

UNIVERSITÉ PARIS-DAUPHINE – PSL
CENTRE DE RECHERCHE EN MATHÉMATIQUES DE LA DÉCISION

**REINFORCEMENT LEARNING METHODS APPLIED TO
ATARI GAME BASIC MATH**

AXELLE MERIC

COURSE SUPERVISOR:

YATING LIU

ASSISTANT PROFESSOR (MAÎTRE DE CONFÉRENCES) - CEREMADE



REINFORCEMENT LEARNING
ACADEMIC YEAR 2025/2026

1 Environment Description

Reinforcement Learning (RL) has demonstrated remarkable success in solving complex tasks, notably in the Atari 2600 domain. However, while most Atari games require quick reflexes, Atari Basic Math presents a fundamentally cognitive challenge. In this environment utilising the Gymnasium Arcade Learning Environment (ALE), an arithmetic equation is displayed at the top of the screen, and the agent must navigate a cursor to select the correct answer among several numbers scattered across the grid.

To apply RL algorithms, we formally define the Basic Math environment as a Markov Decision Process (MDP) defined by the tuple $(S, \mathcal{A}, \mathcal{R})$:

State Space (S): Instead of processing raw RGB pixels, which would necessitate computationally expensive Convolutional Neural Networks (CNNs), we extract the RAM (Random Access Memory) state of the Atari console. The state space is represented by an array of 128 variables ($s \in [0, 255]^{128}$). This poses a major dimensionality challenge, as the theoretical state space size is 256^{128} . It is represented by the space `Box(0, 255, (128,), uint8)`. This abstraction allows us to use lighter function approximation methods, while retaining the complete internal state of the game.

Action Space ($\mathcal{A}$): The action space is `Discrete(6)`. The agent interacts with the environment through a discrete action space of size 6, corresponding to the joystick inputs: $\mathcal{A} = \{\text{NOOP (0), FIRE (1), UP (2), RIGHT (3), LEFT (4), DOWN (5)}\}$. The FIRE action is specifically used to submit an answer.

Reward Function ($\mathcal{R}$): The default reward structure is strictly sparse. The agent receives a reward of +1.0 only when it correctly selects the right answer using the FIRE action, and 0.0 at all other times.

Game Mechanics and Objective: Basic Math is an educational game where the player is presented with algorithmic problems, including addition, subtraction, multiplication, and division. The player must use the joystick directional inputs (Up, Down, Left, Right) to cycle through numbers and select the correct answer, finally pressing the action button (FIRE) to submit it. Correctly solving the equation yields a positive reward.

However, a Reinforcement Learning agent possesses no inherent mathematical knowledge. It does not know how to "read" digits or "compute" an addition. Therefore, the true objective of applying RL to this environment is to train the agent to discover a complex mapping: associating the internal representation of an equation (the RAM state) with the specific sequence of navigation and action inputs that leads to a positive reward. The core challenge of this project is to align the agent's purely mathematical objective (maximising the cumulative reward function via the Bellman equation) with the game's actual educational objective.

Episode Termination Conditions: An episode typically ends when the agent has answered a specific number of questions (done) or when a hard time limit built into the ALE emulator is reached (truncated).

Note that, to ensure the strict independence of our experiments and to avoid the side effects inherent to running a Jupyter Notebook, we took care to instantiate a new, clean environment via `gym.make()` for the evaluation of each algorithm.

2 Baseline Evaluation

Following the progression of the course, we initially approached the Atari Basic Math environment using classical Reinforcement Learning methods. Our primary goal was to solve the MDP using standard Q-Learning as seen in the lectures. However, attempting to apply these

foundational methods directly to an Atari environment quickly revealed severe limitations.

2.1 Tabular Q-Learning on Basic Environment

We first implemented a standard Tabular Q-Learning agent. Due to the immense size of the state space, maintaining a traditional NumPy array for the Q-Table would immediately result in an Out-Of-Memory error. To circumvent this, we utilised a dynamic Python dictionary that initialises state-action values on the fly.

The agent was trained on the default environment with the sparse reward structure. The results showed a complete failure to learn. Because the $+1.0$ reward is virtually impossible to stumble upon through purely random exploration, the Q-values remained at zero. Consequently, during exploitation phases, the agent's behavior collapsed into repeating a single useless action (predominantly NOOP), proving that the sparse reward is insufficient to guide early learning.

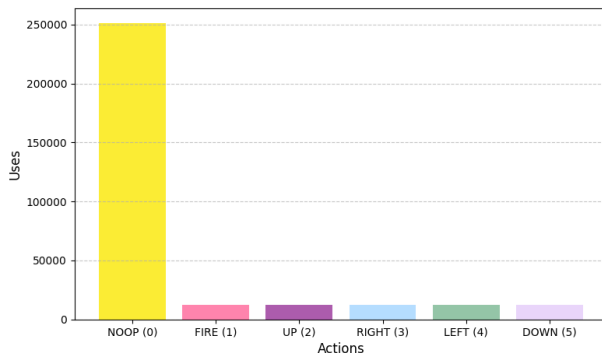


Figure 1: Action Distribution by Q-Table

2.2 Reward Shaping: Time and Action Penalties

Upon analysing the tabular agent's failure on the base environment, we hypothesised that the strictly sparse $+1.0$ reward provided insufficient guidance. To encourage active exploration and discourage trivial behaviors, we implemented two forms of Reward Shaping via custom Environment Wrappers:

- **Time Penalty:** A small, constant negative reward applied at every step (e.g., $r = -0.01$). This penalty acts as a "living cost," incentivising the agent to solve the problem as quickly as possible rather than wandering indefinitely or getting stuck in a NOOP loop.
- **Action Penalty (FIRE):** A specific penalty ($r = -1.0$) inflicted whenever the agent executes the FIRE action without successfully solving the equation. This heavily discourages the agent from randomly spamming the submission button to farm the terminal state.

2.3 Tabular Q-Learning with Shaped Rewards

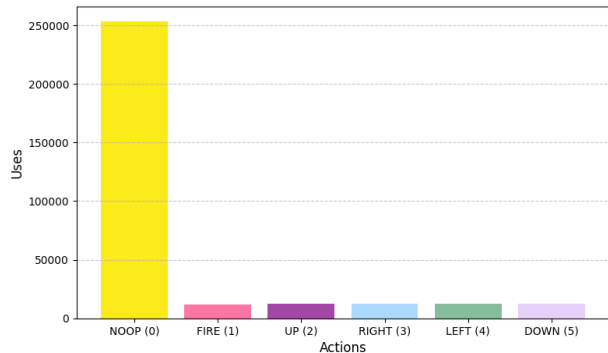


Figure 2: Action Distribution by Q-Table with Penalty

We retrained the Tabular Q-Learning agent on the newly wrapped environment to observe if a penalised reward signal could facilitate learning. However, as illustrated in the action distribution histogram (Figure 2), the shaped rewards completely failed to improve the agent's behavior. Instead of actively exploring the grid, the agent overwhelmingly stagnated on the NOOP action (Action 0).

This absolute failure occurs because the tabular agent suffers from the curse of dimensionality. With the Atari RAM state continuously updating, the agent constantly encounters "novel" states. Since it cannot generalise across similar

states, its Q-table remains overwhelmingly initialised at zero. When all estimated Q-values for a given state are zero, the argmax policy simply defaults to the first available index, which is Action 0 (NOOP). Furthermore, the introduction of penalties may induce "freezing" behavior, where the agent avoids taking meaningful actions to prevent accumulating negative rewards.

The fact that tabular methods collapse into a NOOP loop, regardless of how meticulously the rewards are shaped, undeniably proves that solving Atari Basic Math requires a robust function approximator. This mathematical necessity dictates our transition to Deep Reinforcement Learning.

3 Deep Reinforcement Learning: DQN Implementation

3.1 Network Architecture, Replay Buffer and DQN Agent

Network Architecture To replace the exhaustive Q-Table, we utilised PyTorch (`nn.Module`) to construct a Multi-Layer Perceptron (MLP). A crucial pre-processing step inside the network's forward pass is normalising the RAM inputs. By dividing the raw integer values by 255.0, we force the inputs into a $[0, 1]$ range, which prevents exploding gradients and ensures learning stability.

The network processes these normalised inputs through two hidden linear layers (128 neurons each) using ReLU activation functions to capture non-linear mathematical relationships in the environment. Importantly, the final output layer (which predicts the 6 Q-values) does not use an activation function. This is necessary because expected Q-values can be mathematically negative (e.g., anticipating a penalty), and applying a ReLU would incorrectly truncate these vital negative signals to zero.

Experience Replay Buffer If a neural network learns sequentially from consecutive game frames, it suffers from catastrophic forgetting, overfitting to a specific trajectory while entirely forgetting past lessons. To break this temporal correlation, we implemented an Experience Replay Buffer.

This finite memory queue (capacity of 10,000) stores transitions (S_t, A_t, R_t, S_{t+1}) . During training, the agent randomly samples mini-batches from this buffer. This random sampling ensures that the gradient descent steps are independent and identically distributed (i.i.d), drastically stabilising the optimisation process.

The DQN Agent and Learning Dynamics

The agent integrates the network and the buffer to execute its learning logic, specifically addressing two major RL challenges:

- **Exploration vs. Exploitation:** The agent utilises an ϵ -greedy policy. Initially, $\epsilon = 1.0$, forcing 100% random actions to discover the environment. After each episode, this rate is decayed (here multiplied by 0.995) until it reaches a floor of 0.01, shifting the agent towards exploiting the network's learned knowledge while maintaining a marginal exploratory behavior.
- **The Moving Target Problem:** Standard Q-Learning updates rely on the network's own predictions to calculate the Bellman target. To prevent a destabilising feedback loop, we instantiate two identical networks. The Main Network actively learns, while the Target Network remains frozen to provide stationary mathematical targets, updating its weights only periodically.

3.2 Computational Optimisation: Frame Skipping

Initial training runs revealed a severe computational bottleneck due to the 60 frames-per-second nature of the Atari environment. Processing a forward and backward pass for every micro-frame is highly redundant.

To resolve this, we implemented a Custom Environment Wrapper to apply **Frame Skipping** (Action Repeat). The agent now selects an action once every 4 frames, and this action is automatically repeated across the intermediate steps.

To accurately evaluate the impact of this optimisation, we plotted the learning curves over 150 episodes (Figure 3). Because RL environments exhibit extremely high variance from one episode to the next due to the ϵ -greedy exploration, plotting raw rewards results in illegible, noisy data. Therefore, we applied a Moving Average with a window size of 20 episodes to filter out the statistical noise and reveal the underlying learning trend.

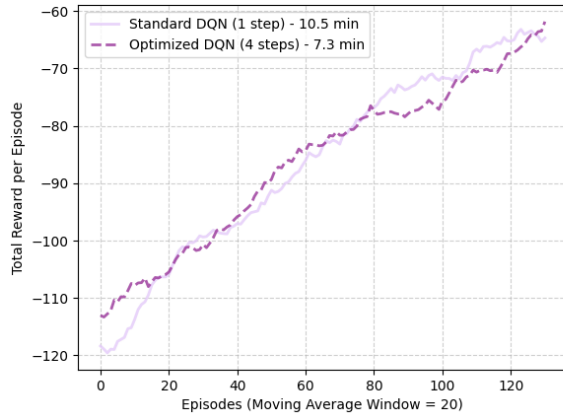


Figure 3: Impact of Frame Skipping on learning performance

As illustrated, the Optimised DQN (4 steps) achieved near-identical convergence to the Standard DQN while reducing the training time from 10.5 minutes to 7.3 minutes, proving the viability of this technique.

4 Reward Shaping and Behavioral Analysis

With the computational bottleneck resolved via Frame Skipping, we evaluated the agent's ability to learn from the environment's native sparse rewards. In the Atari BasicMath environment, the agent only receives a +1.0 reward for submitting the correct mathematical answer,

and 0.0 otherwise. As hypothesised, this sparse signal proved insufficient to guide the Deep Q-Network effectively. To address this, we implemented and analysed various Reward Shaping strategies.

4.1 Initial Baselines and The "Fear of FIRE" Phenomenon

We initially compared the baseline sparse reward against two shaping strategies: a constant Time Penalty (-0.01 per step) to encourage speed, and a severe Action Penalty (-1.0) applied every time the agent incorrectly used the FIRE action (Action 1).

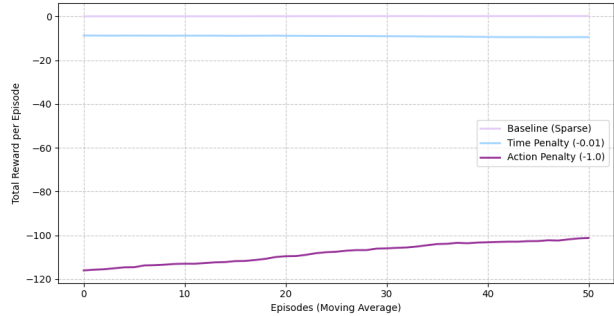


Figure 4: Reward Shaping Comparison on DQN Agent

While the Baseline and Time Penalty configurations failed to yield any meaningful learning, the Action Penalty of -1.0 forced a radical change in the learning curve. However, this change was pathological. By analysing the agent's learned action distribution, we observed a phenomenon we term the "Fear of FIRE".

Because the penalty for a mistake (-1.0) mathematically offsets the reward for a success ($+1.0$), the risk-to-reward ratio became highly unfavorable. To maximise its expected return (or rather, to minimise its losses), the agent converged on a risk-averse, paralysed policy. It learned to exclusively use NOOP or directional movements, completely avoiding the submission button. The network essentially learned that "not playing" was the mathematically safest strategy to survive, a classic example of Reward Hacking.

4.2 Hyperparameter Tuning via Ablation Study

To cure this paralysis, the penalty needed to be finely balanced: it had to be negative enough to discourage mindless spamming of the FIRE button (which breaks the learning process), but small enough that the expected value of discovering the true goal (+1.0) outweighs the risk of a few mistakes.

To find this optimal equilibrium, we conducted an ablation study, testing four distinct penalty values for the FIRE action: -1.0 (Severe), -0.5 (Moderate), -0.1 (Mild), and 0.0 (Baseline).

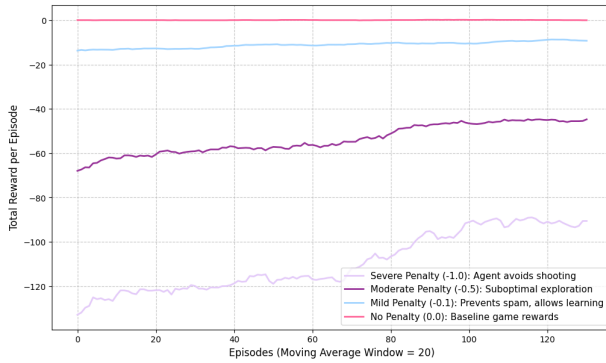


Figure 5: Impact of Action-Specific Reward Shaping on Deep Q-Network

Quantitative analysis of the final training phases (average reward over the last 50 episodes), which gives Figure 5 provides empirical evidence of this behavioral shift. With the severe penalty (-1.0), the agent achieved a highly negative average score of -91.06 , reflecting its total paralysis and the accumulation of penalties during forced exploration. Halving the penalty to -0.5 showed a partial recovery, bringing the average to -45.51 , though the exploration remained heavily constrained.

However, the mild penalty (-0.1) yielded a dramatic ten-fold improvement, raising the asymptotic average reward to -9.55 . Furthermore, the training logs indicate an increase in computational time for the -0.1 configuration (10.8 minutes compared to ~ 7.1 minutes for stricter penalties). This time variance suggests that the agent in the -0.1 environment is actively engaging with the game mechanics and evaluat-

ing deeper trajectories, rather than falling into a rapid, repetitive loop of "safe" actions like Action 2 (UP).

Based on this empirical evidence, the Mild Penalty (-0.1) combined with Frame Skipping ($\text{skip}=4$) constitutes our final, optimised Deep RL architecture for this environment.

5 Comparative Analysis: Deep Q-Network vs. Deep SARSA vs. Deep Monte Carlo

To definitively validate our optimised architecture, we conducted an extensive, long-run training session spanning 1500 episodes. Because training Deep Reinforcement Learning agents on Atari environments is exceedingly slow and computationally expensive (taking several hours per algorithm), we serialised all episodic logs, rewards, and execution times into .json files. This engineering choice guarantees reproducibility and allows for instantaneous comparative plotting without the need to re-execute the prohibitively long training loops. We benchmarked our tuned Deep Q-Network (Off-Policy TD) against Deep SARSA (On-Policy TD) and Deep Monte Carlo (On-Policy Episodic). All three agents utilised the exact same neural network capacity, Frame Skipping ($\text{skip} = 4$ which was directly implemented in the Agent initialisation), and the optimised Reward Shaping penalty (-0.1 for incorrect FIRE actions).

5.1 Reward Convergence and Computational Dynamics

As illustrated in Figure 6, the long-run comparative analysis reveals a fundamental limitation. Despite our rigorous optimisations, none of the algorithms successfully solved the environment.

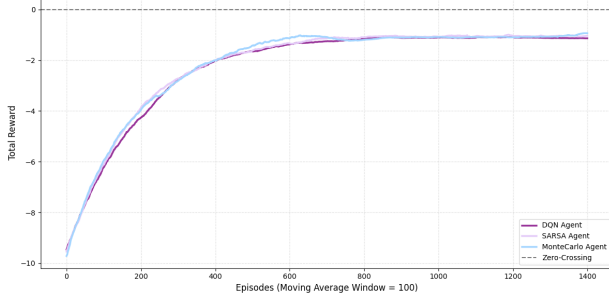


Figure 6: Performance Comparison (Total Reward) over 1500 Episodes

The learning curves for all three agents overlap almost perfectly, converging towards the same negative asymptote without ever crossing the zero-reward baseline. This uniform failure across both Off-Policy (DQN) and On-Policy (SARSA, MC) methods strongly suggests that the bottleneck lies not in the learning algorithms themselves, but in the state representation.

The Atari BasicMath environment requires the agent to visually read an equation, calculate the result, and select the correct answer. Relying strictly on the 128-byte RAM observation space presents a severe "curse of dimensionality". The mapping between these raw memory integers and the semantic meaning of the mathematical problem is highly non-linear. Standard Multi-Layer Perceptrons (MLPs) simply cannot extract these hidden features within a 1500-episode budget. Consequently, all agents eventually settle into sub-optimal, risk-averse policies to minimise the -0.1 penalties, abandoning the true $+1.0$ objective.

5.2 Action Distribution: The Universal "Fear of FIRE"

The root cause of this collective failure is glaringly visible in the agents' action distributions (Figure 7). Despite our efforts to mitigate the penalty (setting it to a mild -0.1), all three algorithms ultimately succumbed to the same pathological behavior: they completely stopped using the FIRE button.

By analysing the evolutionary plots of the actions over time, we observe that any initial

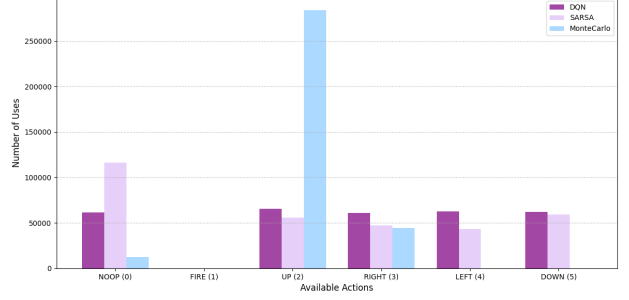


Figure 7: Action Distribution (Over the last 50 episodes) for all three architecture

exploration of the FIRE action is quickly suppressed. Because the agents fail to bridge the logical gap between the obscure 128-byte RAM state and the $+1.0$ sparse reward, almost every press of the FIRE button is strictly experienced as a -0.1 penalty. Consequently, the networks rapidly update their policies to avoid Action 1 entirely. Deep Q-Network, Deep SARSA, and Deep Monte Carlo all converge on a strategy of perpetual avoidance, defaulting instead to safe directional movements or NOOP.

5.3 Computational Dynamics: Evidence of Time-Limit Truncation

This universal "Fear of FIRE" is perfectly corroborated by the cumulative computation time (Figure 8). As training progresses, the computation time does not plateau; rather, the episodes become increasingly longer for all agents.

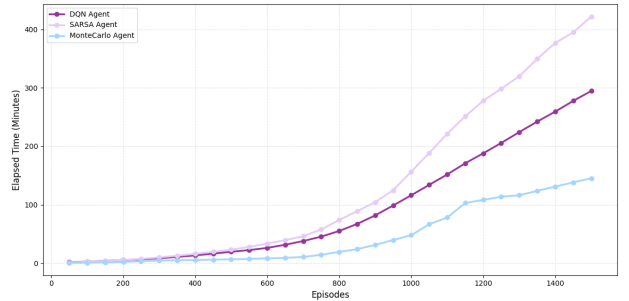


Figure 8: Cumulative Computation Time per Algorithm

In the Atari BasicMath environment, an

episode naturally concludes when the agent submits an answer using the FIRE action. Because our agents learned to be terrified of this button, they never voluntarily submit an answer. Therefore, the episodes only end when the maximum step limit is reached and the environment forcefully aborts the game (triggering the truncated flag in Gymnasium). The increasing slope in the time comparison graph is not a sign of the agents playing well, but rather the empirical proof of their paralysis. The neural networks discovered a mathematical loophole: the optimal way to minimise the -0.1 action penalty in an incomprehensible environment is to do absolutely nothing and simply wait for the game’s internal timer to run out.

6 Live Diagnostics and Hypothesis Elimination

Following the long-run comparative analysis, which demonstrated a universal plateau across all algorithms, we conducted a live diagnostic test to visually observe the trained agent’s behavior within the environment.

6.1 Live Environment Testing and Behavioral Observation

We deployed the fully trained agent developed in the previous section in a rendered instance of the Atari BasicMath environment, restricting the execution to a strict limit of 250 steps to prevent infinite looping.

The visual observation and the resulting execution logs perfectly confirmed our quantitative deductions:

```
Test finished after 250 steps.
Total Reward accumulated: 0.00
Result: Episode was forcibly truncated
due to time limit (Agent paralysis).
```

The agent actively navigates the screen using directional inputs but strictly refuses to use

the FIRE button to submit an answer. Consequently, the game never naturally resolves, and the episode is forcibly truncated by our time limit, resulting in a null cumulative score.

This behavior highlights a severe disconnect. The most highly probable root cause is the opacity of the 128-byte RAM observation space. Extracting the semantic logic of a mathematical equation (e.g., $4 + 7 = ?$) and linking it to the spatial position of a cursor purely from raw memory integers is a highly non-linear task that heavily bottlenecks standard Multilayer Perceptrons (MLPs).

Ideally, the optimal solution would be to discard the RAM state and train a Convolutional Neural Network (CNN) directly on the raw RGB pixels, allowing the agent to "read" the screen spatially. However, training CNNs on Atari environments requires massive computational resources and tens of millions of frames—far exceeding the hardware and time constraints of this study.

Therefore, since we cannot alter the fundamental observation space, we must proceed by elimination. In the following section, we will test the remaining accessible components of the environment to definitively rule out other potential sources of algorithmic failure. During the live test observation, we identified a critical behavioral nuance, as captured in Figure 9.



Figure 9: Live test observation

As the screenshot illustrates, the agent success-

fully navigates the cursor to the correct answer. This visual evidence yields a fascinating hypothesis: the neural network may have actually learned to solve the equations (successfully mapping the 128-byte RAM state to the correct cursor position), but suffers from an extreme "Fear of FIRE". The -0.1 penalty, initially designed to prevent action spamming, seems to have created an insurmountable barrier, causing the agent to prefer a zero-reward timeout over the perceived risk of a penalty.

To rigorously validate this hypothesis, we will proceed with two targeted experiments by isolating the Monte Carlo architecture:

1. **Penalty Ablation (Section 6.2):** We will test the MC agent under two new penalty constraints (-0.01 and no penalty) to determine if a safer environment can naturally break this paralysis.
2. **Forced FIRE Mechanism (Section 6.3):** We will implement a custom environment wrapper that programmatically forces the agent to trigger the FIRE action after a set number of steps. By overriding its fear, we can objectively measure if its underlying positional policy is indeed accurate.

6.2 Penalty Hypothesis elimination

To rigorously test the hypothesis that our agent had secretly learned the mathematical logic but was paralysed by the -0.1 action cost, we conducted a penalty differential study. We trained separate Deep Monte Carlo agents with varying FIRE penalties: 0.0 (No Penalty), -0.01 (Minimal Penalty) to compare with the previously used -0.1 penalty.

As shown in Figure 10 and Figure 11, the magnitude of the penalty strictly dictates the agent's behavioral collapse, but in different ways.

Remarkably, even a minimal penalty of -0.01 is sufficient to induce severe risk aversion. The agent still completely abandons Action 1 (FIRE) and resorts to looping safe movements (like UP) to await the episode timeout. The "Fear of FIRE"

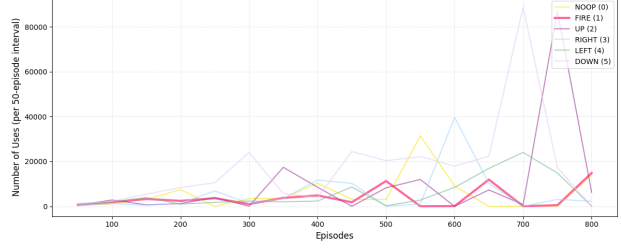


Figure 10: Evolution of Action Selection over Time without penalty

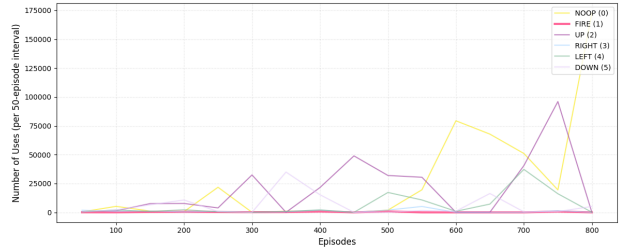


Figure 11: Evolution of Action Selection over Time for a -0.01 penalty

is deeply entrenched in the TD/MC optimisation process.

When the penalty is completely removed (0.0), the fear vanishes, but it is not replaced by intelligence. As seen in the 0.0 penalty curve, the agent does begin to use the FIRE button, but its usage is highly erratic. There is no discernible positive learning trend indicating a correlation between the math equation and the cursor placement. Instead, the agent simply reverts to a baseline of random guessing, as further evidenced by the flattened action distribution (Figure 12).

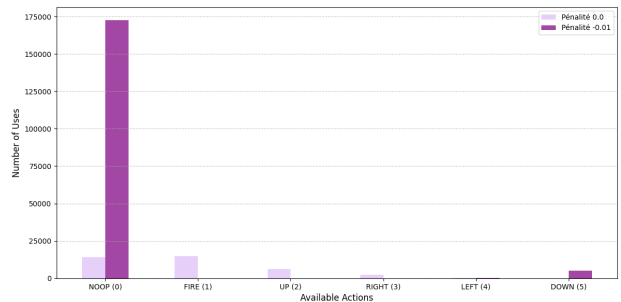


Figure 12: Action Distribution (Over the last 50 episodes)

The definitive proof of this failure lies in the reward trajectory (Figure 13).

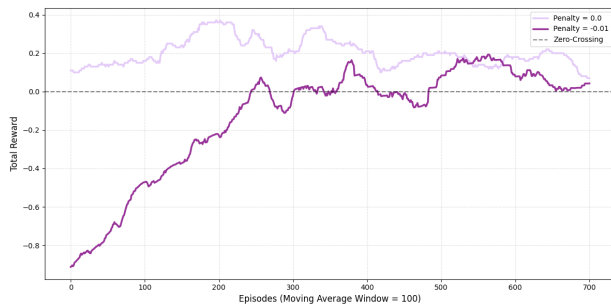


Figure 13: Performance Comparison (Total Reward) over 800 Episodes

Even when free to shoot without consequence, the unpenalised agent’s reward fluctuates randomly around zero. This ablation study conclusively refutes the optimistic hypothesis. The agent was not hiding a correct mathematical understanding behind its fear; rather, the penalty merely masked the fact that it was completely lost. The Multi-Layer Perceptron architecture remains fundamentally incapable of extracting semantic meaning from the raw 128-byte RAM state.

6.3 Absolute "Fear of FIRE" Hypothesis

Having established that a lack of penalty leads to random guessing, we conducted one final diagnostic to completely isolate the agent’s spatial and logical understanding. We engineered a custom wrapper for the Deep Monte Carlo agent (with a -0.01 penalty) that programmatically forces the agent to execute the FIRE action exactly at step 250. Immediately following this forced action, the episode is artificially terminated.

This strict formulation guarantees perfect credit assignment: the final reward (whether a $+1.0$ victory or a -0.01 penalty) is directly tied to wherever the agent had navigated the cursor over the preceding 249 steps.

As expected, the action distribution (Figure 14 and Figure 15) reflects this constraint. The FIRE action represents only a microscopic fraction of

the total actions, as it is strictly limited to one execution per episode, while the agent uses its movement keys to position the cursor before the forced trigger.

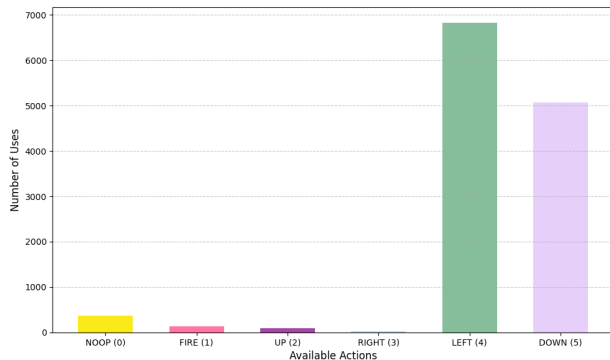


Figure 14: Action Distribution for Forced Monte-Carlo

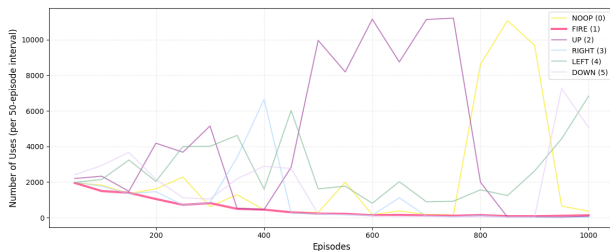


Figure 15: Evolution of actions over time for Forced Monte-Carlo

If the neural network had successfully learned the mapping between the RAM variables, the equation, and the cursor position, this forced mechanism would bypass its psychological "fear" and force it to collect the $+1.0$ victory rewards. However, the reward trajectory (Figure 16) provides the definitive disproof of this hypothesis.

Despite being forced to shoot, the agent’s reward asymptotes tightly around -0.02 . It consistently misses the correct answer, accumulating only movement penalties and the final incorrect answer penalty.

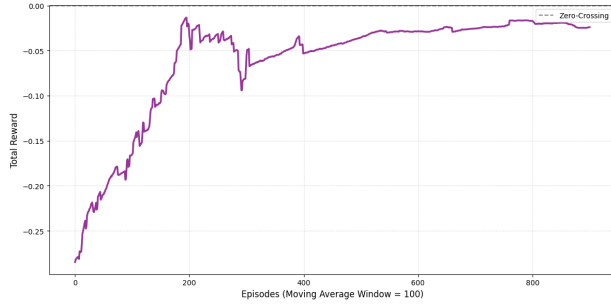


Figure 16: Reward Performance of Forced Monte-Carlo

7 General Conclusion

This comprehensive study aimed to evaluate and optimise Deep Reinforcement Learning algorithms—specifically Deep Q-Network, Deep SARSA, and Deep Monte Carlo; on the complex, logic-driven environment of Atari BasicMath. Through rigorous hyperparameter tuning, reward shaping, and extensive long-run training across 1500 episodes, we attempted to bridge the gap between simple spatial navigation and cognitive mathematical reasoning.

Despite these extensive optimisations, the primary finding of this research is a universal algorithmic plateau. None of the evaluated architectures successfully solved the environment or reliably crossed the zero-reward baseline.

By conducting a methodical root-cause analysis; including live visual diagnostics, penalty ablation studies, and forced action mechanisms; we systematically eliminated alternative explanations for this failure. While we cannot claim to have isolated the root cause with absolute certainty, the empirical evidence strongly exonerates the learning algorithms themselves and the reward shaping mechanics. Consequently, our leading and most robust hypothesis is that the fundamental bottleneck lies within the state representation.

The 128-byte RAM observation space presents an insurmountable "curse of dimensionality" for standard Multi-Layer Perceptrons within a standard computational budget. Extracting the semantic meaning of a mathematical equation (e.g., $3 + 4 = ?$) and linking it causally to the

spatial coordinates of a cursor purely from raw memory integers is a highly non-linear task that eludes basic Deep TD and MC methods. Without an explicit understanding of the rules, the agents naturally regress to risk-averse strategies, exploiting time-limit truncations to avoid penalties: a fascinating, albeit unintended, display of reward hacking.

To conquer environments that demand explicit reading and logical deduction, future iterations must transition away from RAM states. Employing Convolutional Neural Networks (CNNs) on raw RGB pixel observations would allow the agent to spatially "read" the screen, mirroring human cognitive processing. Coupled with significantly larger computational budgets (tens of millions of frames) and advanced intrinsic motivation algorithms, this pixel-based approach represents the definitive next step toward solving Atari BasicMath.

Ultimately, this project highlights a crucial tenet of Artificial Intelligence: an algorithm is only as powerful as the state representation it is given to understand the world.